
CORSO DI ARCHITETTURE DEGLI ELABORATORI A.A. 2025/2026
PROF.SSA VALENTINA CASOLA

ISTRUZIONI IN ASSEMBLY ARM

Dr. Alessandra Somma
alessandra.somma@unina.it

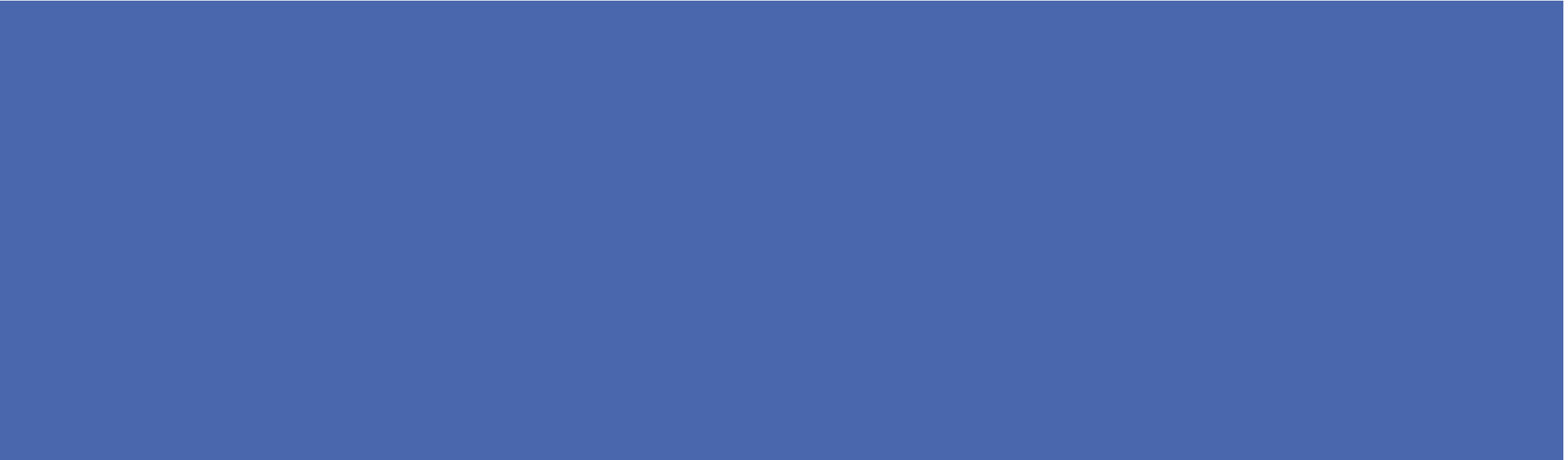
CATEGORIE DI ISTRUZIONI IN ARM

- Istruzioni di elaborazione dati
- Istruzione di trasferimento dati
- Controllo del flusso di esecuzione
- Subroutine e programmazione modulare



ISTRUZIONI DI ELABORAZIONE DATI

ASSEGNAZIONE DI VALORI, ISTRUZIONI ARITMETICHE, ISTRUZIONI DI CONFRONTO, ISTRUZIONI BOOLEANE



ISTRUZIONI DI ASSEGNAZIONE VALORI (1)

- La sintassi assembly ARM GAS per istruzioni di assegnazione di valori (**MOV** e **MVN**) è:

MOV{cond}{S} Rd, Operand2

MVN{cond}{S} Rd, Operand2

- dove:
 - cond è un suffisso opzionale per l'esecuzione condizionale.
 - S è il suffisso opzionale che indica al processore di settare i flag nel registro CPSR.
 - Rd è un **registro** indicato come **destinazione** dell'operazione di copia
 - Operand2 è **l'operando sorgente** dell'operazione di copia, di tipo registro o costante numerica specificata attraverso il modo di indirizzamento immediato.
- Entrambe le istruzioni **lavorano solo sul tipo word da 32 bit.**

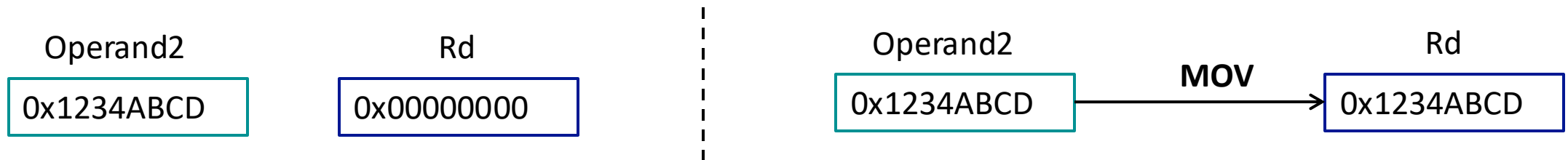
ISTRUZIONI DI ASSEGNAZIONE VALORI (2)

- La sintassi assembly ARM GAS per istruzioni di assegnazione di valori (**MOV** e **MVN**) è:

MOV{cond}{S} Rd, Operand2

MVN{cond}{S} Rd, Operand2

- L'istruzione **MOV** copia il valore dell'operando sorgente nel registro destinazione.



- L'istruzione **MVN** copia il valore dell'operando sorgente, negato bit a bit, nel registro destinazione.



ISTRUZIONI DI ASSEGNAZIONE VALORI (3)

- La sintassi assembly ARM GAS per istruzioni di assegnazione di valori (**MOV** e **MVN**) è:

MOV{cond}{S} Rd, Operand2

MVN{cond}{S} Rd, Operand2

- Operand2 può essere:
 - Un **valore costante** (immediato con #).
 - Il valore costante non viene codificato su 32 bit direttamente all'interno dell'istruzione in linguaggio macchina (per ridurre la dimensione dell'istruzione). In realtà, deve essere codificato come ROR(Imm, 2*Rotate), dove Imm è un campo di 8 bit (0-255) e Rotate è un campo di 4 bit (0-15) → il **valore costante deve essere un byte (0-255) ruotato a destra di un numero pari di bit (0,2,4 .. 30)**.
 - Se in assembly usiamo una costante non rappresentabile come (Imm, Rotate) (per esempio MOV R0, 0x12345678), l'assemblatore da errore → **USO DI PSEUDOISTRUZIONE LDR** (da non confondere con l'istruzione LDR)
 - Un **registro del processore**, eventualmente effettuando un'operazione di **scorrimento** di un certo numero di posizioni delle cifre binarie in esso contenute.

PSEUDOISTRUZIONI DI SCORRIMENTO (1)

- La sintassi assembly ARM GAS per **pseudoistruzioni di scorrimento** è:

```
LSL{cond}{S}  Rd, Rp, #const  
LSR{cond}{S}  Rd, Rp, #const  
ASR{cond}{S}  Rd, Rp, #const  
ROR{cond}{S}  Rd, Rp, #const
```

Pseudoistruzioni non sono delle vere e proprie istruzioni. Sono usate dal programmatore e l'assemblatore le traduce in una o più istruzioni ARM/Thumb (ovvero sono insieme di istruzioni move con l'operand2 shiftato).

- dove:
 - cond è un suffisso opzionale per l'esecuzione condizionale.
 - S è il suffisso opzionale che indica al processore di settare i flag nel registro CPSR.
 - Rd è un **registro destinazione**.
 - #const è la **costante** che indica il **numero di posizioni da scorrere**.
 - Rp è un **registro sorgente**.

ISTRUZIONI DI ARITMETICA (1): ADDIZIONE

- La sintassi assembly ARM GAS per istruzioni di aritmetica (somma) è:

ADD{cond}{S} Rd, Rn, Operand2

ADC{cond}{S} Rd, Rn, Operand2

ADD esegue operazione di aritmetica su operandi a 32 bit.
ADC somma due operandi a 32 bit ed il valore del flag C (riporto).

- dove:
 - cond è un suffisso opzionale per l'esecuzione condizionale.
 - S è il suffisso opzionale che indica al processore di settare i flag nel registro CPSR.
 - Rd è un **registro destinazione** dell'operazione di somma.
 - Rn è un **registro sorgente** dell'operazione di somma costituendo il primo addendo.
 - Operand2 è **l'operando sorgente** dell'istruzione e contiene il secondo addendo (può essere sia di tipo registro sia di tipo immediato)
- Se al codice mnemonico si aggiunge il suffisso S il processore setta i flag N, Z, C, e V nel registro CPSR in accordo con il risultato. Ad esempio ADDS R0, R1, R2 calcola la somma R1+R2 e mette il risultato in R0. A seconda del risultato salvato in R0, i flag dei registri CPSR vengono settati opportunamente.

ISTRUZIONI DI ARITMETICA (2): SOTTRAZIONE

- La sintassi assembly ARM GAS per istruzioni di aritmetica (sottrazione) è:

SUB{cond}{S} Rd, Rn, Operand2
SBC{cond}{S} Rd, Rn, Operand2

SUB esegue operazione di aritmetica su operandi a 32 bit. **SBC** somma due operandi a 32 bit ed il valore del flag C (che viene considerato come un prestito).

- dove:
 - cond è un suffisso opzionale per l'esecuzione condizionale.
 - S è il suffisso opzionale che indica al processore di settare i flag nel registro CPSR.
 - Rd è un **registro destinazione** dell'operazione di sottrazione.
 - Rn è un **registro sorgente** dell'operazione di sottrazione (costituendo il minuendo).
 - Operand2 è **l'operando sorgente** dell'istruzione e contiene il sottraendo (può essere sia di tipo registro sia di tipo immediato)
- La SUB calcola $Rd = Rn + (-Operand2)$
- La SBC calcola $Rd = Rn + (-Operand2) - NOT(C)$

ISTRUZIONI DI ARITMETICA (3): MOLTIPLICAZIONE

- La sintassi assembly ARM GAS per istruzioni di aritmetica (moltiplicazione) è:

MUL{cond}{S} Rd, Rn, Rs

- dove: **MLA**{cond}{S} Rd, Rn, Rs, Rm

- cond è un suffisso opzionale per l'esecuzione condizionale.
- S è il suffisso opzionale che indica al processore di settare i flag nel registro CPSR.
- Rd è un **registro destinazione** dell'operazione di sottrazione.
- Rn è un **registro primo operando sorgente** diverso da Rd.
- Rs è un **registro** che funge **da secondo** operando **sorgente**;
- Rm è un **registro** che funge **da terzo** operando **sorgente (accumulatore)**;
- L'istruzione MUL calcola il prodotto su 32 bit ($Rn * Rs$) e pone il risultato nel registro Rd.
- L'istruzione MLA calcola il prodotto su 32 bit ($Rn * Rs$) e lo somma al contenuto del registro Rm, ponendo il risultato nel registro Rd. Rm può coincidere con Rd.
- N.B** Se Rn è usato come registro destinazione (Rn uguale a Rd) il risultato è imprevedibile!

ISTRUZIONI DI TEST

- La sintassi assembly ARM GAS per istruzioni di test (**TST** e **TEQ**) è:

TST{cond} Rn, Operand2

TEQ{cond} Rn, Operand2

- dove:
 - cond è un suffisso opzionale per l'esecuzione condizionale.
 - Rn è un **registro primo operando sorgente**.
 - Operand2 è un secondo **operando sorgente** dell'istruzione (che può essere di tipo registro o di tipo immediato).
- L'istruzione TST calcola **l'and bit a bit** tra il valore contenuto in Rn e il valore di Operand2. **I flag del registro di stato vengono alterati** in base al risultato dell'operazione. **Il risultato dell'operazione non è memorizzato.** Ad esempio TST R0, #0x0C fa sì che il flag Z del CPSR sia 1 se i bit 2 e 3 del registro R0 sono entrambi 0, altrimenti il flag vale 0.
- L'istruzione TEQ esegue un'operazione booleana xor (**or-esclusivo**) **bit a bit** tra il valore Rn e il valore di operand2. Questa operazione è utile per verificare l'uguaglianza del contenuto del registro Rn con il valore di Operand2. A differenza della CMP, la TEQ non ha l'effetto di modificare anche il flag di carry C. Il risultato dell'operazione xor non è memorizzato.

ISTRUZIONI DI COMPARAZIONE

- La sintassi assembly ARM GAS per istruzioni di comparazione (**CMP** e **CMN**) è:

CMP{cond} Rn, Operand2
CMN{cond} Rn, Operand2

- dove:
 - cond è un suffisso opzionale per l'esecuzione condizionale.
 - Rn è un **registro primo operando sorgente**.
 - Operand2 è un secondo **operando sorgente** dell'istruzione (che può essere di tipo registro o di tipo immediato).
- L'istruzione CMP calcola (Rn-Operand2) e setta i flag (del CPSR) conseguentemente.
- L'istruzione CMN calcola (Rn-(-Operand2)) e setta i flag (del CPSR) conseguentemente.

CODICI DI CONDIZIONE E SIGNIFICATO

- In tante istruzioni, abbiamo visto la voce {cond}.
 - {cond} è una forma generica per indicare uno dei possibili mnemonici che indicano la **condizione**.

<i>cond</i>	Mnemonic	Name	CondEx
0000	EQ	Equal	Z
0001	NE	Not equal	$\bar{Z}$
0010	CS / HS	Carry set / Unsigned higher or same	C
0011	CC / LO	Carry clear / Unsigned lower	$\bar{C}$
0100	MI	Minus / Negative	N
0101	PL	Plus / Positive of zero	$\bar{N}$
0110	VS	Overflow / Overflow set	V
0111	VC	No overflow / Overflow clear	$\bar{V}$
1000	HI	Unsigned higher	$\bar{Z}C$
1001	LS	Unsigned lower or same	$Z OR \bar{C}$
1010	GE	Signed greater than or equal	$\bar{N} \oplus \bar{V}$
1011	LT	Signed less than	$N \oplus V$
1100	GT	Signed greater than	$\bar{Z}(\bar{N} \oplus \bar{V})$
1101	LE	Signed less than or equal	$Z OR (N \oplus V)$
1110	AL (or none)	Always / unconditional	ignored

ESEMPIO DI MODIFICA DEI FLAG DEL REGISTRO DI STATO (1)

- **Esempio 1:** Istruzione di comparazione CMP

CMP R5, R6

- Questa istruzione calcola R5-R6, non salva il risultato e setta i flag.
 - Se il risultato è zero $\rightarrow Z=1$
 - Se il risultato è negativo $\rightarrow N = 1$
 - Se il risultato causa un riporto $\rightarrow C = 1$
 - Se il risultato causa un overflow $\rightarrow V = 1$

ESEMPIO DI MODIFICA DEI FLAG DEL REGISTRO DI STATO (2)

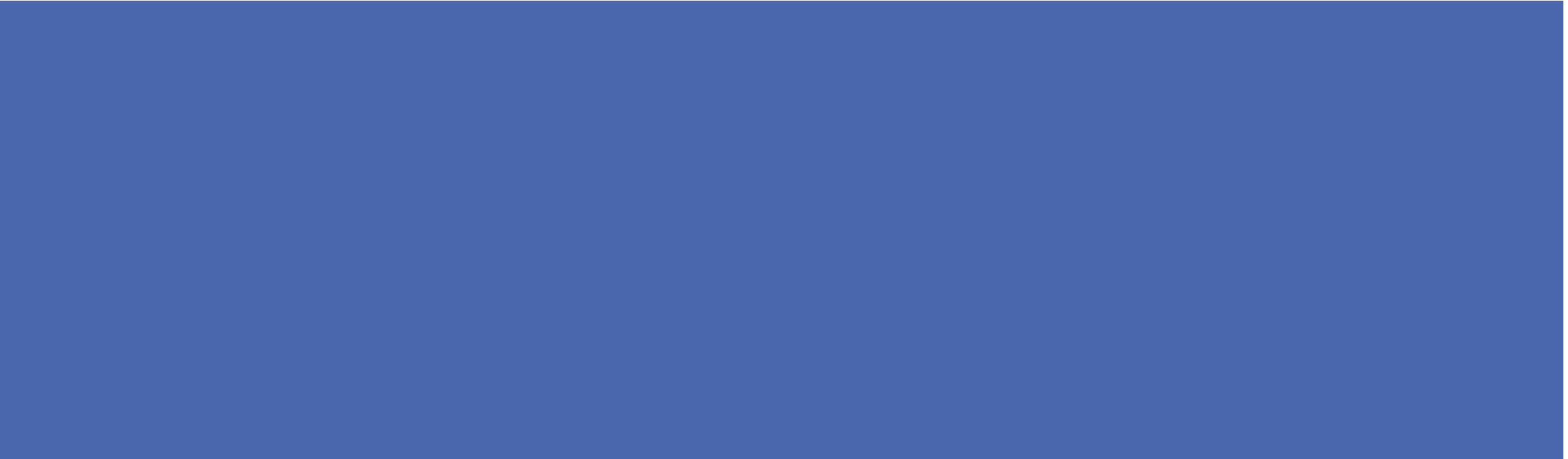
- **Esempio 2:** aggiungere il suffisso S al codice mnemonico di un'istruzione per settare i flag di condizione:

ADD**S** R1, R2, R3

- Calcola $R2 + R3$
- Data la presenza del suffisso S, setta i flag del registro di stato: se il risultato è 0 ($Z=0$), se il risultato è negativo ($N=1$) e così via.
- Salva il risultato in R1.

ISTRUZIONE DI TRASFERIMENTO DATI

ISTRUZIONI LDR E STR, MODI DI INDIRIZZAMENTO, PSEUDO-ISTRUZIONE LDR



ISTRUZIONI LDR E STR

- La sintassi assembly ARM GAS per istruzioni di trasferimento dati da-verso la memoria è:

LDR {cond}	R_dest, Op_src
STR {cond}	R_src, Op_dst

- dove:
 - LDR ha come primo operando (R_dest) un registro ed è operando destinazione.
 - STR ha come primo operando (R_src) un registro ed è operando sorgente.
 - Gli operandi Op_src, Op_dst delle operazioni LDR e STR rispettivamente sono **operandi di tipo memoria**.
 - L'uso del suffisso S per le istruzioni LDR e STR non è consentito.
 - Entrambe le istruzioni possono essere opzionalmente associate ad una condizione booleana cond, espressa tramite codici mnemonici indicati in tabella, che subordina l'esecuzione dell'istruzione alla verità della condizione stessa.

MODI DI INDIRIZZAMENTO DI OPERANDI IN MEMORIA

- La determinazione **dell'indirizzo dell'operando memoria**, che prende il nome di **effective address (EA)**, può avvenire in diversi modi, a seconda del **modo di indirizzamento** usato.
- I diversi modi di indirizzamento possibili per l'operando di tipo memoria differiscono per il modo con cui ne determinano l'indirizzo, e per se e come alterano il valore dei registri che sono utilizzati per determinare l'EA.
- Per le istruzioni LDR e STR, l'effective address dell'operando in memoria **deve essere allineato ad un indirizzo multiplo di quattro**, cioè il suo valore in binario deve avere i due bit meno significativi pari a 0.
- I modi di indirizzamento in ARM sono:
 - Indirizzamento indiretto
 - Indirizzamento indiretto con spiazzamento costante/da registro
 - Indirizzamento indiretto con post-indicizzazione costante/da registro
 - Indirizzamento indiretto con pre-indicizzazione costante/da registro

MODO DI INDIRIZZAMENTO INDIRETTO

- Nel *modo di indirizzamento indiretto*, l'indirizzo della locazione di memoria è specificato in un registro Rn con la seguente sintassi:

LDR{cond} R_dest, [Rn]
STR{cond} R_src, [Rn]

- L'effective address è il valore contenuto nel registro Rn.

EA = [Rn]

- Il contenuto del registro Rn non è alterato da questo modo di indirizzamento.

MODO DI INDIRIZZAMENTO CON SPIAZZAMENTO COSTANTE

- Nel *modo di indirizzamento con spiazzamento costante* (nel manuale chiamato *immediate offset*), il codice dell'istruzione specifica un registro Rn, una costante offset che può assumere un valore tra 0 e 4095 e un segno S (+1/-1):

LDR{cond} R_dest, [Rn, #+/-offset]
STR{cond} R_src, [Rn, #+/-offset]

- **L'effective address è calcolato come segue:**

$$EA = Rn + S * offset$$

- Il contenuto del registro **Rn non è alterato da questo modo di indirizzamento.**

MODO DI INDIRIZZAMENTO CON SPIAZZAMENTO DA REGISTRO

- Nel *modo di indirizzamento indiretto* con **spiazzamento da registro** (nel manuale chiamato *register offset*), il codice dell'istruzione specifica due registri Rn ed Rm.

LDR{cond} R_dest, [Rn, Rm]
STR{cond} R_src, [Rn, Rm]

- Il registro Rn contiene un indirizzo base mentre il registro Rm contiene uno spiazzamento (offset). L'effective address è calcolato come segue:

$$EA = Rn + Rm$$

- Il contenuto dei registro **Rn** e **Rm** non è alterato da questo modo di indirizzamento.

MODO DI INDIRIZZAMENTO CON POST-INDICIZZAZIONE COSTANTE

- Nel *modo di indirizzamento indiretto* con **post indicizzazione costante** (nel manuale chiamato *immediate post-indexed*), il codice dell'istruzione specifica un registro Rn, una costante offset che può assumere un valore compreso tra 0 e 4095, ed un segno S (+1/-1).

LDR{cond} R_dest, [Rn], #+/-offset
STR{cond} R_src, [Rn], #+/-offset

- L'effective address è il contenuto del registro Rn.

$$EA = Rn$$

- Il contenuto dei registro **Rn** è **alterato da questo modo di indirizzamento**. E' incrementato del valore S*offset **dopo** che il processore lo ha utilizzato per determinare l'effective address.

$$Rn = Rn + S * offset$$

MODO DI INDIRIZZAMENTO CON POST-INDICIZZAZIONE DA REGISTRO

- Nel *modo di indirizzamento indiretto* con **post indicizzazione da registro** (nel manuale chiamato *register post-indexed*), il codice dell'istruzione specifica due registri Rn e Rm. La notazione è:

LDR{cond} R_dest, [Rn], Rm
STR{cond} R_src, [Rn], Rm

- L'effective address è il contenuto del registro Rn.

$$EA = Rn$$

- Il contenuto dei registro **Rn** è **alterato da questo modo di indirizzamento**. E' incrementato del valore Rm, **dopo** che il processore lo ha utilizzato per determinare l'effective address.

$$Rn = Rn + Rm$$

MODO DI INDIRIZZAMENTO CON PRE-INDICIZZAZIONE COSTANTE

- Nel *modo di indirizzamento indiretto con **pre indicizzazione costante*** (nel manuale chiamato *immediate pre-indexed*), il codice dell'istruzione specifica un registro Rn, una costante offset che può assumere un valore compreso tra 0 e 4095, ed un segno S (+1/-1).

LDR{cond} R_dest, [Rn, #+/-offset]!

STR{cond} R_src, [Rn, #+/-offset]!

- Il contenuto dei registro **Rn** è **alterato da questo modo di indirizzamento**. E' incrementato del valore S*offset **prima** che il processore lo ha utilizzato per determinare l'effective address.

$$\mathbf{Rn} = \mathbf{Rn} + \mathbf{S * offset}$$

- L'effective address è il contenuto del **nuovo valore del registro Rn**.

$$\mathbf{EA} = \mathbf{Rn}$$

MODO DI INDIRIZZAMENTO CON PRE-INDICIZZAZIONE DA REGISTRO

- Nel *modo di indirizzamento indiretto* con **pre indicizzazione da registro** (nel manuale chiamato *register pre-indexed*), il codice dell'istruzione specifica due registro Rn ed Rm.

LDR{cond} R_dest, [Rn, Rm]!
STR{cond} R_src, [Rn, Rm]!

- Il contenuto dei registro **Rn** è **alterato da questo modo di indirizzamento**. E' incrementato del valore Rm **prima** che il processore lo ha utilizzato per determinare l'effective address.

$$\mathbf{Rn} = \mathbf{Rn} + \mathbf{Rm}$$

- L'effective address è il contenuto del **nuovo valore del registro Rn**.

$$\mathbf{EA} = \mathbf{Rn}$$

ISTRUZIONI PER TRASFERIMENTO DI WORD, HALFWORD, BYTE

- Le istruzioni LDR ed STR lavorano sulle word (parole a 32 bit, quindi 4 byte). L'ISA ARM è dotata anche di varianti delle istruzioni di **load e store atte a trasferire dati di tipo byte (8 bit) o halfword (16 bit)** che occupano in memoria una o due locazioni consecutive, rispettivamente.

LDRB{cond} R_dest, Op_src

STRB{cond} R_src, Op_dest

LDRH{cond} R_dest, Op_src

STRH{cond} R_src, Op_dest

- Le istruzioni LDRB e STRB lavorano su un dato di 8 bit che in memoria occupa una singola locazione. Le istruzioni LDRH e STRH lavorano su un dato di 16 bit che in memoria occupa due locazioni consecutive.
- Op_src e Op_dest sono operandi di tipo memoria.
- Sia per LDRB che per LDRH la restante parte del registro R_dst è riempita con zeri.** Questo significa considerare unsigned il dato copiato dall'istruzione.

PSEUDO-ISTRUZIONE LDR

- Nel linguaggio ARM GAS, il codice mnemonico LDR (da non confondere con l'istruzione LDR!!) indica anche una pseudo-istruzione, mediante la quale il programmatore può caricare in un registro un qualsiasi valore costante rappresentabile su 32 bit.

LDR{cond} R_dest, =const
LDR{cond} R_dest, =label

- dove:
 - cond è il codice opzionale per l'esecuzione condizionale;
 - R_dst è un registro indicato come destinazione dell'operazione.
 - const è un valore costante rappresentato con 32 bit;
 - label è un simbolo (etichetta) definita altrove nel programma.
- Perché una pseudo-istruzione? L'assemblatore **sostituisce**, al momento della traduzione, **la pseudo-istruzione LDR con l'istruzione o la sequenza di istruzioni** più opportune da ottenere per lo scopo.
 - L'assemblatore sostituirà la LDR proprio con un'istruzione di MOV/MOVN e tecniche come la constant pool, ma in questo modo il programmatore non dovrà preoccuparsi di gestire la problematica dell'operand nella MOV.

La pseudo-istruzione LDR è molto utile per spostare un qualsiasi valore in un registro poiché il trasferimento dati con l'istruzione MOV richiederebbe di un operando in modo che possa esser codificato come (Imm, Rotate).

ESERCIZIO 1: SOMMA DI DUE VARIABILI E SALVATAGGIO RISULTATO IN MEMORIA

- **Esercizio 1:** somma due variabili a e b allocate in memoria.

codice in C

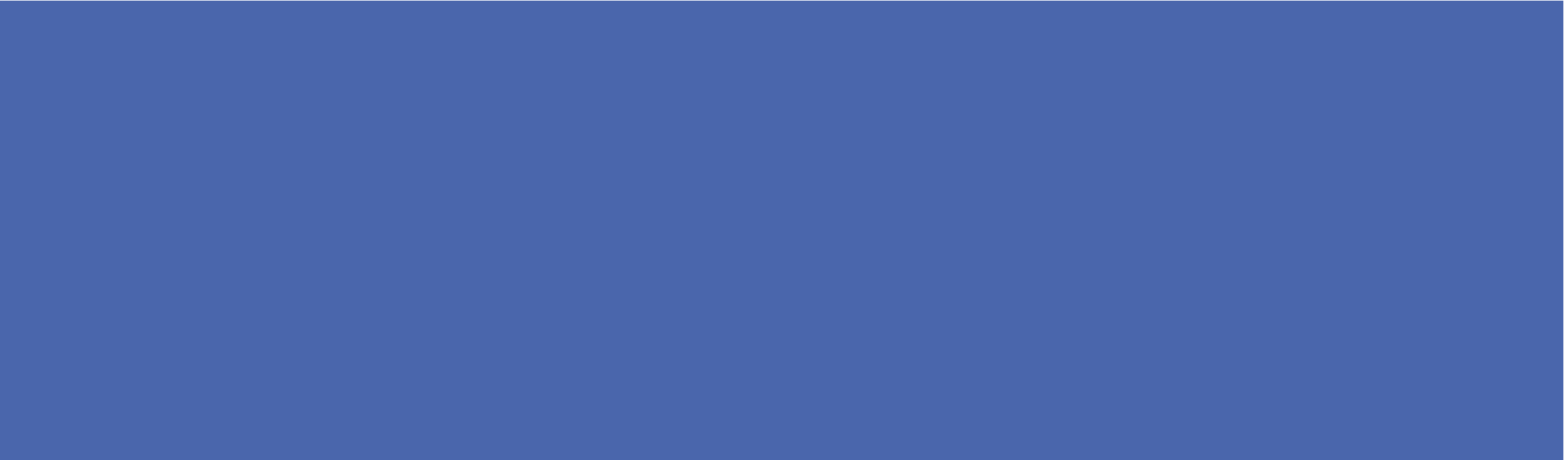
```
int a = 10;  
int b = 20;  
int res;  
  
res = a+b;
```

Assembly ARM

```
.text  
.global _start  
  
_start:    LDR R0, =a  
           LDR R1, [R0]  
           LDR R0, =b  
           LDR R2, [R0]  
           ADD R3,R1,R2  
           LDR R0, =res  
           STR R3,[R0]  
           SWI 0x11  
  
           .data  
a:         .word 10  
b:         .word 20  
res:       .word 0
```

ISTRUZIONI DI CONTROLLO DEL FLUSSO DI ESECUZIONE

ESECUZIONE CONDIZIONALE, ISTRUZIONI DI SALTO, COSTRUTTI DI SELEZIONE ED ITERAZIONE



CONTROLLO DEL FLUSSO DI ESECUZIONE

- Il controllo del flusso di esecuzione serve per **alterare**, appunto, **l'esecuzione sequenziale del flusso**: non vogliamo che il processore esegua sempre il codice in maniera sequenziale.
- Per esempio:
 - *Costrutti if/else, costrutti while*: solo se si verifica una determinata condizione, quel blocco di codice sarà eseguito.
 - *Branching*: saltare ad un'altra porzione di codice, se una condizione è vera.
- Abbiamo già visto come in assembly ARM siano presenti i **flag di condizione** che possono essere:
 - settati da un'istruzione (ad esempio si esegue un'operazione di sottrazione il cui risultato è nullo, questo comporta la modifica del flag Z del registro di stato).
 - usati per **esecuzione condizionale** di un'istruzione.

ESEMPIO DI ESECUZIONE CONDIZIONALE (1)

- **Esempio 3:** aggiungere il codice di condizione di esecuzione al codice mnemonico di un'istruzione per eseguire l'istruzione in maniera condizionale

```
CMP      R1, R2
SUBNE    R3, R5, R8
```

- **NE** è il codice mnemonico per la condizione Not Equal.
- L'esecuzione della sottrazione è condizionata al risultato dell'operazione precedente. La compare andrà ad eseguire il confronto tra il registro R1 ed R2, se i due registri sono uguali ($R1-R2=0$) l'istruzione ha l'effetto di settare il flag Z a 1, altrimenti se i registri sono diversi il flag Z sarà 0. **La SUB sarà eseguita solo se R1 è diverso da R2 (il flag Z settato in seguito all'esecuzione dell'istruzione CMP è 0).**

ESEMPIO DI ESECUZIONE CONDIZIONALE (2)

CMP	R5, R9
SUBEQ	R1, R2, R3
ORRMI	R4, R0, R9

- Supponiamo che $R5 = 17$ e $R9 = 23$
 1. CMP esegue $17 - 23 = -6 \rightarrow$ i flag del registro di stato sono settati a : $Z = 0$; $N = 1$; $C = 0$; $V = 0$
 2. SUBEQ non esegue l'operazione di sottrazione tra R2 ed R3, perché è condizionata al fatto che R5 ed R9 siano uguali. Siccome $R5 \neq R9$ e quindi $Z = 0$, la sottrazione non viene eseguita.
 3. ORR esegue se il risultato dell'operazione precedente (la CMP in questo caso, dato che la SUB non ha eseguito) è Minus/Negative, ovvero se il flag N è alto. Siccome l'ultima istruzione eseguita (la CMP) ha settato $N=1$, la ORR esegue l'operazione usando R9 come maschera di R0 (i bit di R9 che sono alti indicano le posizioni dei bit di R0 di cui fare il toggle) e salvando il risultato in R4.

ISTRUZIONI DI SALTO CONDIZIONATO

- Le **istruzioni di salto (branching)** consentono al programmatore di controllare il flusso di esecuzione, in modo da implementare ad esempio costrutti di selezione e iterazione.
- Queste istruzioni agiscono **modificando** a tempo di esecuzione il **valore del registro Program Counter**. Le due istruzioni di salto sono:

B{cond} label

BL{cond} label

- Entrambe le istruzioni saltano all'istruzione di etichetta label al verificarsi della condizione {cond} → il PC viene aggiornato per puntare all'indirizzo dell'istruzione etichettata come label (modo di indirizzamento con spiazzamento costante). La BL, prima di modificare il PC, salva il valore del PC diminuito di 4 nel registro LR (per il discorso del pipelining).
- Sia B che BL possono essere usate per realizzare **salti condizionati (conditional branching)**, aggiungendo al codice mnemonico una condizione cond tra quelle previste in tabella.

ISTRUZIONI DI SALTO INCONDIZIONATO

- Se la condizione cond è assente del codice mnemonico assembly o uguale ad AL (always), si ottiene il codice mnemonico BAL (Branch Always) che rappresenta un'istruzione di **salto incondizionato (unconditional branching)**.
- Il salto alla label **non è condizionato al verificarsi di una determinata condizione**, ma avviene sempre. Per cui quando il processore esegue l'istruzione salterà sempre all'istruzione etichettata come label.

B label

BAL label

- L'istruzione B, se utilizzata senza alcuna condizione {cond}, è del tutto equivalente a BAL.

COSTRUTTO DI SELEZIONE: IF-THEN

- Un **costrutto di selezione if-then** permette di controllare il flusso di esecuzione di un blocco di istruzioni alla verità di una condizione logica.

Pseudocodice in C

```
if (cond) {  
    do_something;  
}  
next_statement;
```

Assembly ARM

```
CMP    R3, #'Y'  
BNE    next  
MOV     R0, #1  
  
next:  ...
```

- N.B.:** un modo alternativo per scrivere questo blocco di codice sarebbe non utilizzare il salto condizionato, ma rendere l'istruzione MOV un'istruzione condizionata (MOVEQ in questo caso). Questa cosa è conveniente quando il blocco di istruzioni «do_something» è piccolo, perché per ogni istruzione del blocco andrà indicata la condizione. Q

COSTRUTTO DI SELEZIONE: IF-THEN-ELSE

- Un **costrutto di selezione if-then-else** serve ad eseguire uno tra due **blocchi di istruzioni in alternativa**, a seconda che una determinata condizione logica risulti vera o falsa.

Pseudocodice in C

```
if (cond) {  
    do_something;  
} else {  
    do_something_else;  
}  
next_statement;
```

Assembly ARM

```
        CMP    R3, #'Y'  
        BEQ    if  
else:    MOV    R0, #2  
        BAL    next  
if:      MOV    R0, #3  
next:    ...
```

COSTRUTTO DI ITERAZIONE: WHILE

- Un **costrutto di iterazione while** serve ad eseguire una sequenza di istruzioni (**corpo del ciclo**) finché una certa espressione booleana *expr* (condizione di permanenza) risulti vera (*true*). La verità dell'espressione è verificata **prima** di eseguire il corpo del ciclo, pertanto il numero di iterazioni può essere di zero o più.
- È *compito del programmatore* inserire nel corpo del ciclo una sequenza di istruzioni che modifichino il contenuto di verità dell'espressione *expr*. Laddove *expr* sia espressa in modo tale da essere sempre vera, il programma produrrebbe un ciclo infinito.

Pseudocodice in C

```
init;  
while (expr) {  
    body;  
    update;  
}  
next_statement;
```

Assembly ARM

```
while:  MOV      R0,#0  
        CMP    R0,#10  
        BGE    next  
        ...  
        ADD    R0,R0,#1  
        BAL    while  
next:   ...
```

COSTRUTTO DI ITERAZIONE: DO-WHILE

- Un **costrutto di iterazione do-while** serve ad eseguire una sequenza di istruzioni (**corpo del ciclo**) finché una certa espressione booleana *expr* (condizione di permanenza) risulti vera (*true*). La verità dell'espressione è verificata **dopo** che il corpo del ciclo sia stato eseguito, pertanto viene effettuata almeno un'iterazione.
- Valgono le stesse considerazioni fatte in precedenza.

Pseudocodice in C

```
init;  
do {  
    body;  
    update;  
} while (expr);  
next_statement;
```

Assembly ARM

```
do:      MOV      R0,#0  
        ...  
        ...  
while:   ADD      R0,R0,#1  
        CMP      R0,#10  
        BLT      do  
next:    ...
```

COSTRUTTO DI ITERAZIONE: FOR

- Un **costrutto di iterazione for** serve ad eseguire una sequenza di istruzioni (**corpo del ciclo**) per un certo numero di volte, a condizione che una certa espressione booleana *expr* risulti vera.
- Il costrutto prevede una sequenza di istruzioni di **inizializzazione** (eseguite solo una volta, prima di verificare la verità dell'espressione *expr*) ed una sequenza di istruzioni di **aggiornamento** della condizione di permanenza, eseguite al termine di ogni iterazione, cioè dopo ogni esecuzione del corpo del ciclo.

Pseudocodice in C

```
for (init; expr; update) {  
    body;  
}  
next_statement;
```

Assembly ARM

```
for:    MOV     R0,#0  
        CMP     R0,#10  
        BGE     next  
        ...  
        ADD     R0,R0,#1  
        BAL     for  
next:   ...
```

ESERCIZIO 1: SOMMA ELEMENTI DI UN VETTORE

- **Esercizio 1:** somma gli elementi di un vettore di 5 elementi.

Assembly ARM

pseudocodice in C

```
int v[5] = {1,2,3,4,5};
int sum = 0;

for (i = 0; i<5; i++) {
    sum = sum + v[i];
}
```

```
.text
.global _start

_start:    LDR R0, =v
           MOV R1,#0 @indice i
           MOV R2,#0 @somma
for_loop:  CMP R1,#5 @confronto i con 5 (dim)
           BEQ end_loop
           LDR R3,[R0]
           ADD R2,R2,R3
           ADD R0,R0,#4 @punto al prossimo elemento
           ADD R1,R1,#1 @ i++
           B  for_loop
end_loop:  LDR R0, =sum
           STR R2,[R0]
           SWI 0x11

           .data
v:         .word 1,2,3,4,5
sum:       .word 0
```

ESERCIZIO 2: CREAZIONE DI UN VETTORE

- **Esercizio 2:** dato un vettore A fatto da 5 elementi, creare un vettore B tale che: se l'elemento i-esimo di A è maggiore di 0, l'elemento i-esimo di B è uguale a quell'elemento di A, altrimenti B è uguale a 0.

pseudocodice in C

```
int a[5] = {1,-2,3,-4,5};
int b[5];

for (i = 0; i<5; i++) {
    if (a[i]>=0)
        b[i] = a[i];
    else
        b[i] = 0;
}
```

```
.text
.global _start

_start:
    LDR R0, =a
    LDR R5, =b
    MOV R1,#0    @indice i
for_loop:
    CMP R1,#5    @ if(i==5)
    BEQ end_loop
    LDR R2,[R0]   @a[i]
    CMP R2,#0    @controllo a[i]
    BLT b_zero   @se a[i]<0 → b[i] = 0
    STR R2,[R5]   @(caso a[i]>=0) b[i]=a[i]
    B incrementi
b_zero:
    MOV R3,#0
    STR R3,[R5]  @b[i]=0
incrementi:
    ADD R0,R0,#4 @punto next valore di a
    ADD R5,R5,#4 @punto next valore di b
    ADD R1,R1,#1
    B for_loop
end_loop:
    SWI 0x11

a:      .data
b:      .word 1,2,3,4,5
        .word 0,0,0,0,0
```

ESERCIZIO 2: USO DI MODI INDIR. POST-INDICIZZ.

- **Esercizio 2.1:** dato un vettore A fatto da 5 elementi, creare un vettore B tale che: se l'elemento i-esimo di A è maggiore di 0, l'elemento i-esimo di B è uguale a quell'elemento di A, altrimenti B è uguale a 0. Facciamo uso del modo di indirizzamento post indicizzazione da costante.

```
.text
.global _start

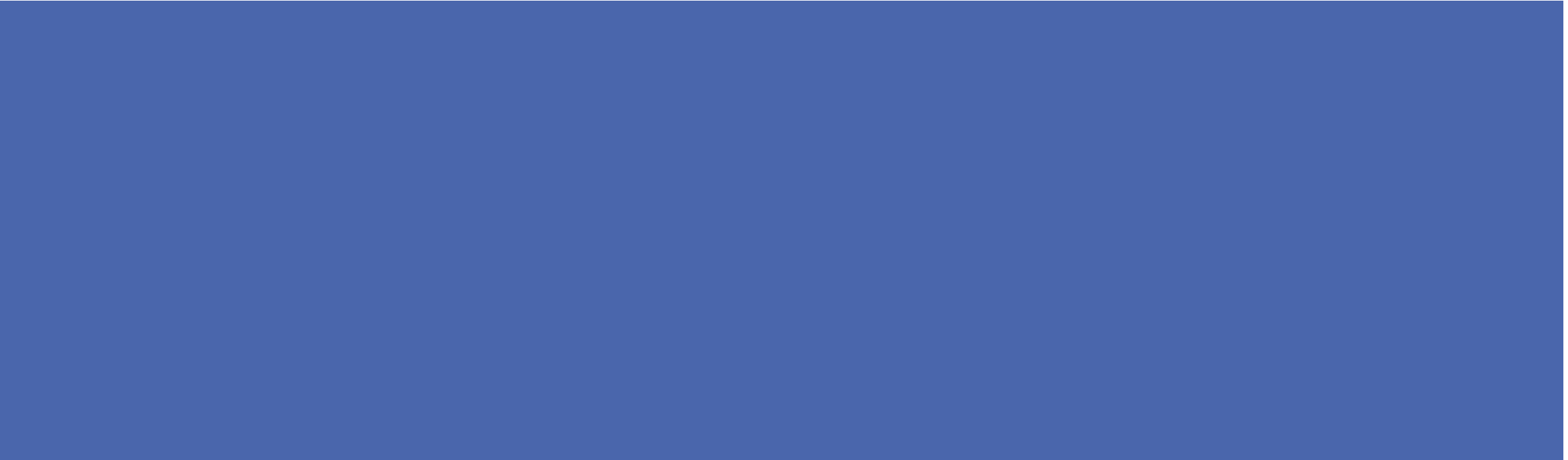
_start:    LDR R0, =a
           LDR R5, =b
           MOV R1, #0    @indice i
for_loop:  CMP R1, #5     @ if(i==5)
           BEQ end_loop
           LDR R2, [R0], #4 @a[i] e incremento il punt.
           CMP R2, #0     @controllo a[i]
           BLT b_zero     @se a[i]<0 → b[i] = 0
           STR R2, [R5], #4 @ (caso a[i]>=0) b[i]=a[i]
           B   incrementi
b_zero:    MOV R3, #0
           STR R3, [R5], #4 @b[i]=0
incrementi: ADD R1, R1, #1
           B   for_loop
           SWI 0x11

end_loop:

.data
a:         .word 1,2,3,4,5
b:         .word 0,0,0,0,0
```

SUBROUTINE E PROGRAMMAZIONE MODULARE

PROGRAMMI E SOTTOPROGRAMMI, PASSAGGIO DI PARAMETRI SU STACK



SUBROUTINE O SOTTOPROGRAMMA

- Con il termine **subroutine** (o **sottoprogramma**) si intende una sequenza di istruzioni che svolge una precisa funzione di elaborazione, la cui esecuzione possa essere invocata (call) all'interno di un programma (programma chiamante). La subroutine è il programma chiamato («invocato»). Il passaggio dal chiamante alla subroutine si realizza attraverso un'istruzione di salto.
- Al termine dell'esecuzione di una subroutine, ci si attende che l'esecuzione del programma chiamante riprenda dall'istruzione successiva a quella che ha determinato l'entrata nella subroutine. Ciò si realizza mediante un'operazione detta «ritorno al programma chiamante» (return), che si realizza con un'istruzione di salto.
- La chiamata a subroutine richiede di gestire due aspetti:
 - **Linkage** (o **collegamento**), ovvero il meccanismo che consente il ritorno al programma chiamante.
 - Lo scambio di informazioni tra **programma chiamante** e **subroutine**.

COLLEGAMENTO DEI SOTTOPROGRAMMI

- In un sistema basato su processore ARM, l'invocazione di una subroutine si effettua con l'istruzione di salto *Branch with Link* BL:

BL{cond} subroutine_entry

- Questa istruzione:
 1. Scrive **l'indirizzo di ritorno** nel registro **Link Register LR** (R14) che è l'indirizzo dell'istruzione del programma chiamante successiva all'istruzione BL di salto a subroutine.
 2. Scrive **l'indirizzo associato all'etichetta** subroutine_entry (entry point) **nel registro PC** (R15).
- Al termine della subroutine il **ritorno al programma chiamante** si effettua copiando l'indirizzo di ritorno da LR in PC con l'istruzione:

MOV **PC,LR**

COLLEGAMENTO DEI SOTTOPROGRAMMI (1)

Esempio in C

```
void main() {  
    int y=0;  
    ...  
    y = sum(42,7);  
    x = y + z;  
    ...  
}
```

```
int sum (int a, int b) {  
    return a+b;  
}
```

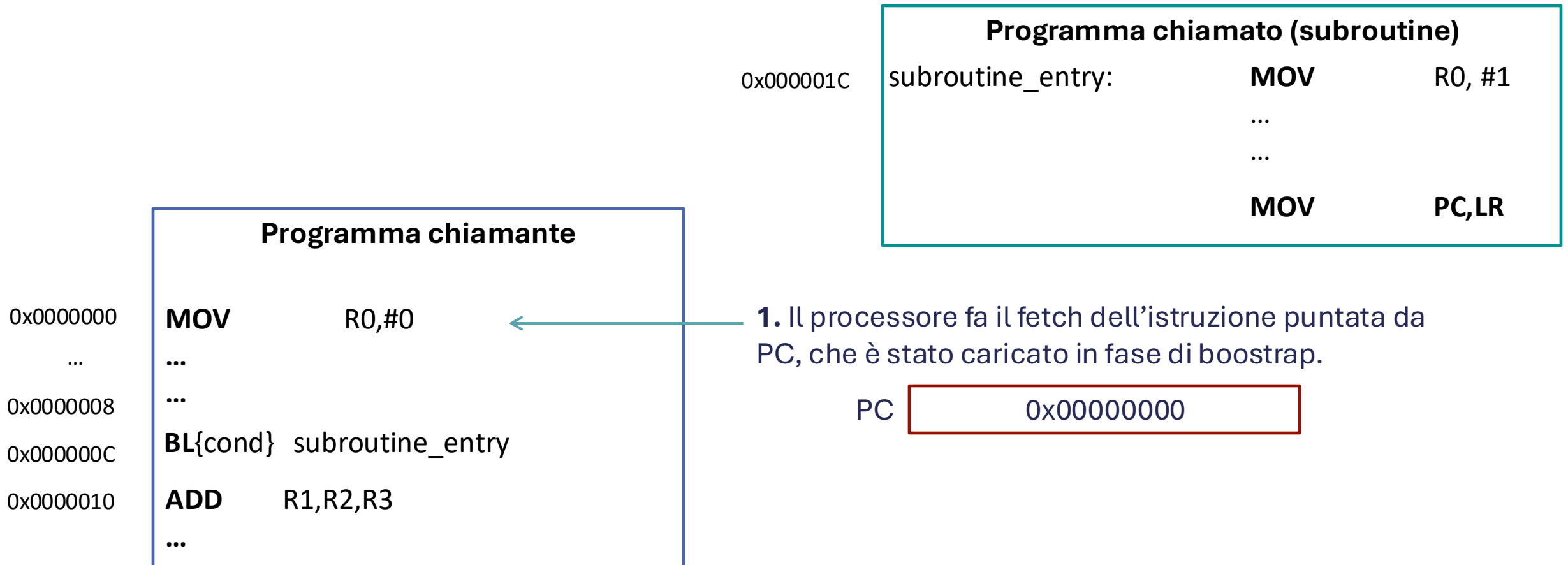
Programma chiamante

```
MOV        R0,#0  
...  
...  
BL{cond} subroutine_entry  
ADD        R1,R2,R3  
...
```

Programma chiamato (subroutine)

```
subroutine_entry:      MOV        R0, #1  
...  
...  
                      MOV        PC,LR
```

COLLEGAMENTO DEI SOTTOPROGRAMMI (2)



COLLEGAMENTO DEI SOTTOPROGRAMMI (3)

0x0000000
...
0x0000008
0x000000C
0x0000010

Programma chiamante

```
MOV      R0,#0  
...  
...  
BL{cond} subroutine_entry  
ADD      R1,R2,R3  
...
```

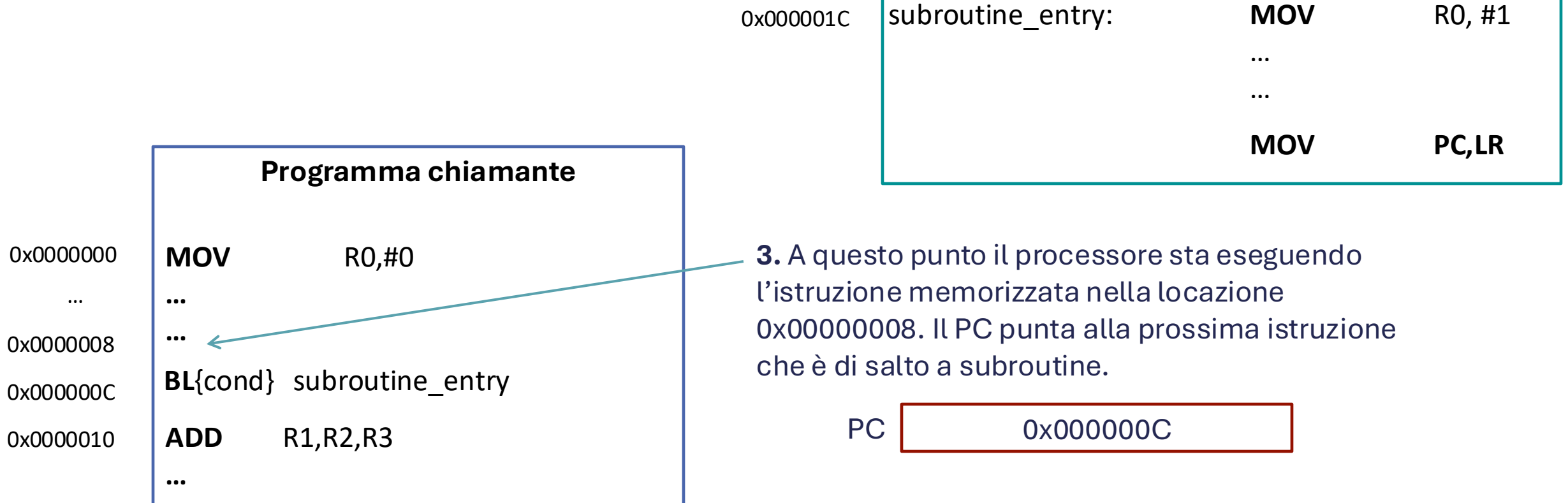
0x000001C

Programma chiamato (subroutine)

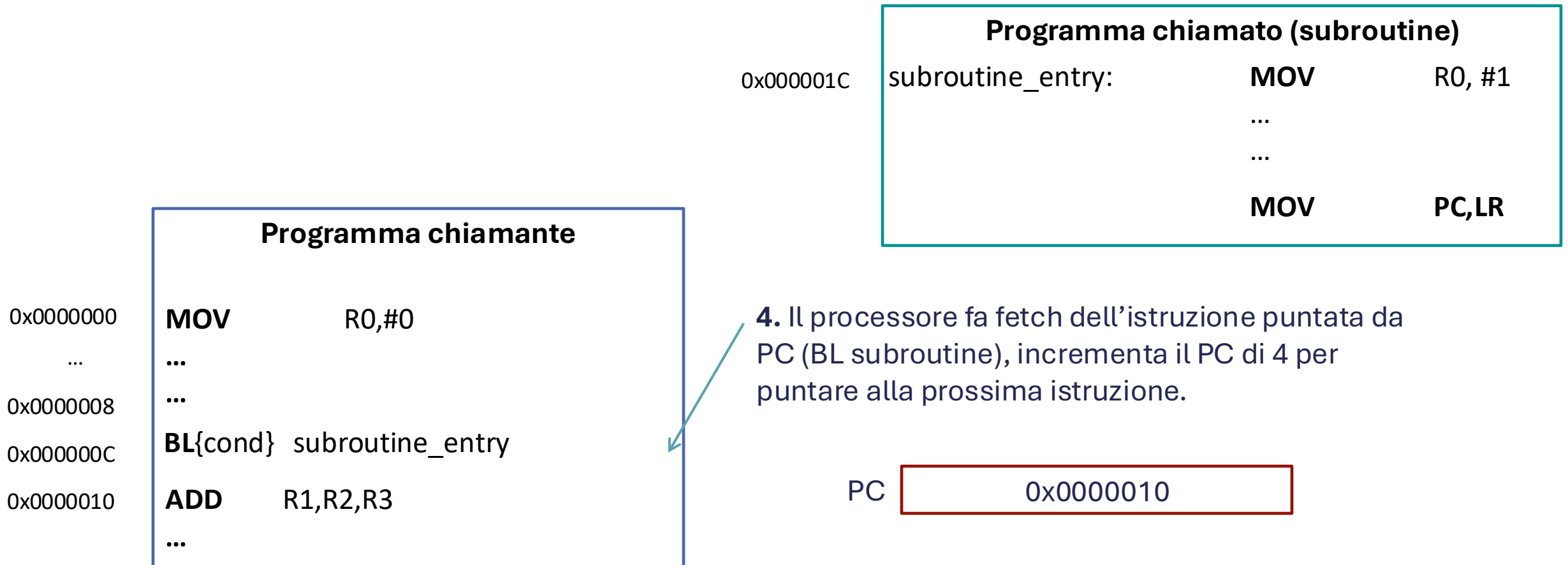
```
subroutine_entry:  MOV      R0, #1  
...  
...  
MOV      PC,LR
```

2. Il processore continua ad eseguire il blocco di istruzioni prima della BL, prelevando, incrementando il PC di 4 dopo ciascuna fetch, decodificando ed eseguendo.

COLLEGAMENTO DEI SOTTOPROGRAMMI (4)



COLLEGAMENTO DEI SOTTOPROGRAMMI (5)



COLLEGAMENTO DEI SOTTOPROGRAMMI (6)

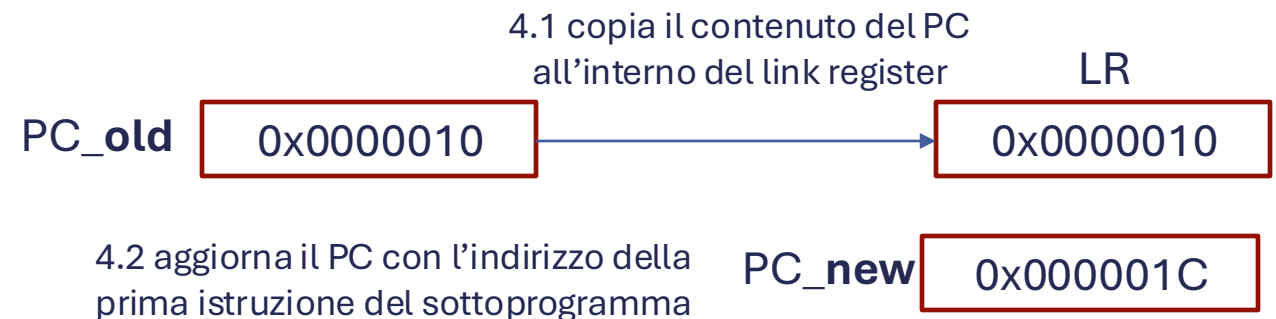
Programma chiamante

```
0x00000000  MOV      R0,#0
...
0x00000008  ...
0x0000000C  BL{cond} subroutine_entry
0x00000010  ADD      R1,R2,R3
...
```

Programma chiamato (subroutine)

```
0x0000001C  subroutine_entry:  MOV      R0, #1
...
...
MOV      PC,LR
```

5. Quando decodifica l'istruzione, si accorge che questa è un'istruzione di salto a subroutine. Se la {cond} è vera, l'istruzione di BL comporta che:



COLLEGAMENTO DEI SOTTOPROGRAMMI (7)

6. Il processore salta all'indirizzo di PC_new che è l'indirizzo della prima istruzione del sottoprogramma e comincia l'esecuzione della subroutine.

0x000001C

Programma chiamante

0x0000000	MOV	R0,#0
...	...	
0x0000008	...	
0x000000C	BL{cond}	subroutine_entry
0x0000010	ADD	R1,R2,R3
	...	

Programma chiamato (subroutine)

subroutine_entry:	MOV	R0, #1
	...	
	...	
	MOV	PC,LR

COLLEGAMENTO DEI SOTTOPROGRAMMI (8)

7. Al termine della subroutine, il processore esegue l'ultima istruzione che **sposta il contenuto del LR all'interno del PC**.

0x000001C

Programma chiamante

```
0x00000000  MOV      R0,#0
...
0x00000008  ...
0x0000000C  BL{cond} subroutine_entry
0x00000010  ADD      R1,R2,R3
...
```

Programma chiamato (subroutine)

```
subroutine_entry:  MOV      R0, #1
...
...
MOV      PC,LR
```

PC_old

0x000000XX

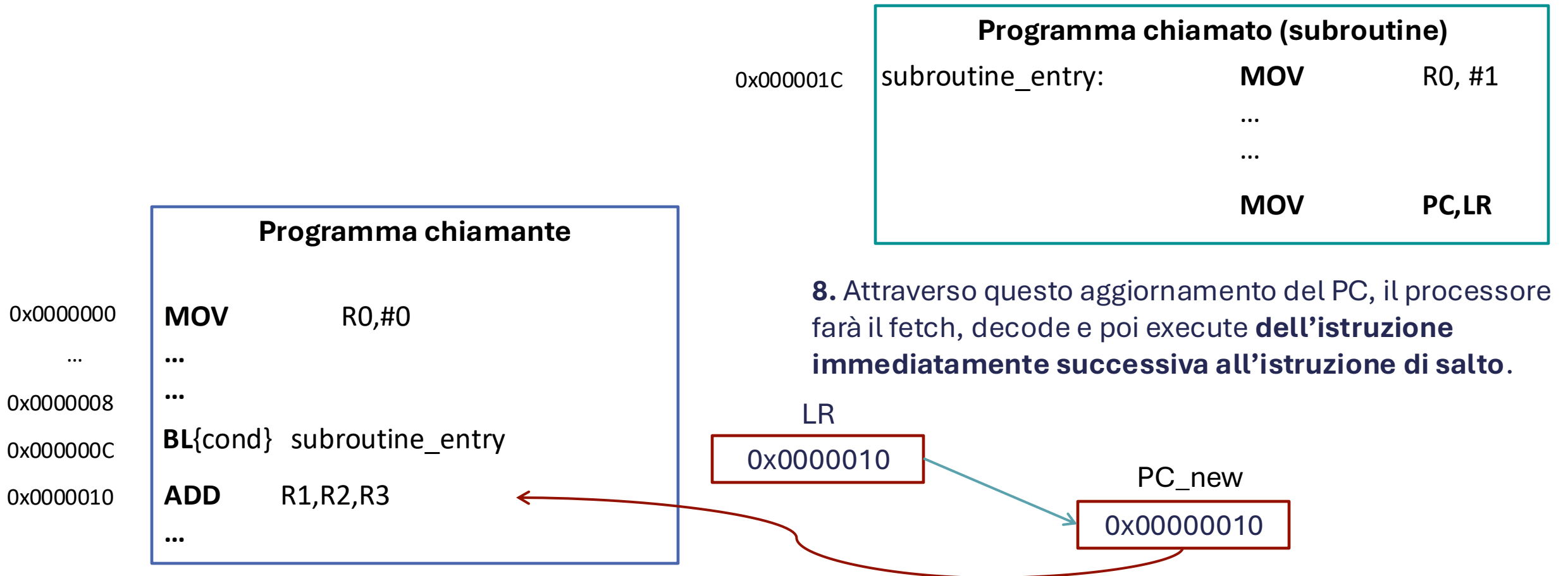
LR

0x00000010

PC_new

0x00000010

COLLEGAMENTO DEI SOTTOPROGRAMMI (9)



PASSAGGIO DI PARAMETRI

- Abbiamo visto come si esegue in assembly la chiamata a sottoprogramma, ma come facciamo ad eseguire il passaggio di parametri tra un programma chiamante ed un sottoprogramma?
- Una subroutine effettua una specifica **elaborazione su dei dati di input che riceve** esplicitamente dal **programma chiamante**, e produce dei **dati di output che sono** esplicitamente **forniti al programma chiamante**.
- I dati di input/output esplicitamente scambiati tra programma chiamante e subroutine costituiscono quello che si chiama **record di attivazione della subroutine**.
- Per un corretto funzionamento dell'interazione tra programma chiamante e subroutine occorre che sia stabilita ed applicata una convenzione circa le modalità con le quali avviene lo **scambio dei valori** dei dati di input e di output.
- Una possibile soluzione per processori ARM che dispongono di svariati registri sarebbe quella di utilizzare i registri del processore come «luogo di scambio» delle informazioni tra chiamante e subroutine. Occorre che il chiamante collochi gli input nei registri previsti dal codice della subroutine, la subroutine collochi gli output nei registri attesi dal programma chiamante.
 - Efficiente perché non richiede accesso in memoria, ma presente effetti collaterali (la subroutine potrebbe produrre variazione sullo stato dei dati del programma chiamante).

PASSAGGIO DI PARAMETRI SU STACK

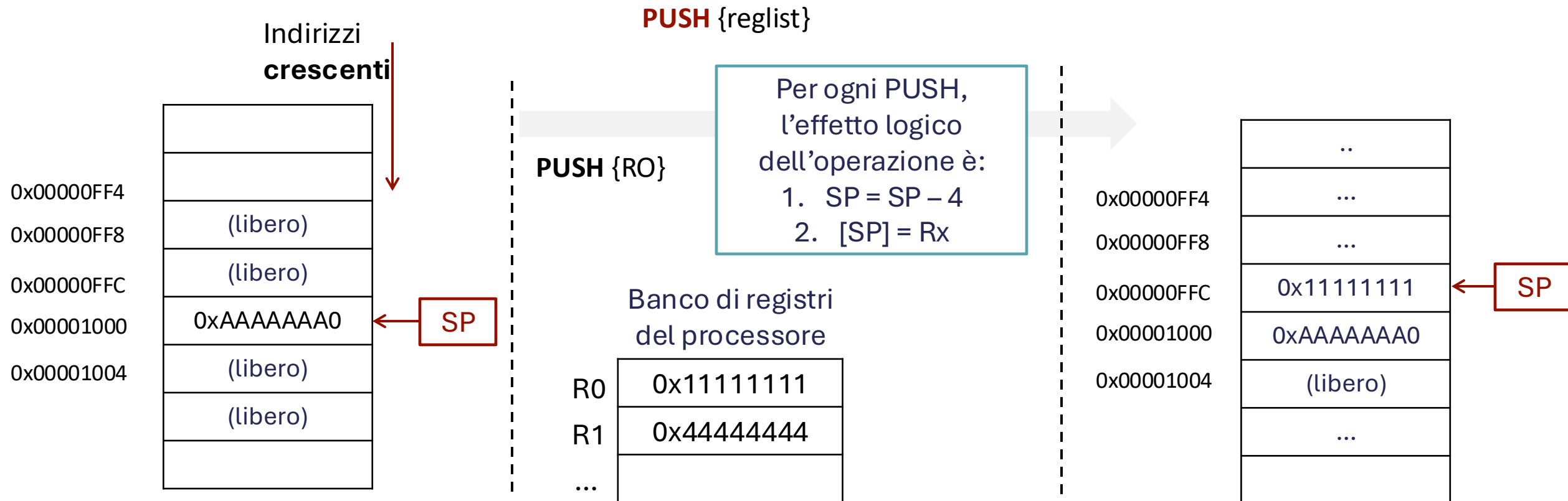
- I parametri di I/O scambiati da una subroutine, organizzati in un record di attivazione, possono essere allocati in un'area di memoria il cui indirizzo è noto sia al programma chiamante sia alla subroutine → **allocazione del record di attivazione sulla cima dello stack puntato dal registro SP**.
- L'allocazione del record di attivazione è compito del programma chiamante, il quale, prima di effettuare la chiamata a subroutine: 1) crea uno spazio vuoto sullo stack per i dati di output, 2) copia sulla cima dello stack i valori dei parametri di input (**push su stack**).
- All'interno della subroutine i **parametri di input/output** saranno accessibili mediante un modo di indirizzamento che ne calcoli **l'indirizzo sommando uno spiazzamento costante all'indirizzo contenuto nel registro SP**.

LO STACK

- Lo **stack** costituisce la **memoria** usata per salvare temporaneamente dati, passare parametri a una funzione, salvare l'indirizzo di ritorno ad una funzione chiamante.
- È organizzato come una pila **last-in-first-out** (LIFO): l'ultimo valore inserito è il primo ad essere tolto.
- Le operazioni che si possono eseguire sullo stack sono:
 - **Push** su stack: caricamento del valore sulla cima dello stack.
 - **Pop** da stack: rimozione del valore dalla cima dello stack.
- Nella convenzione standard ARM, lo stack è **discendente** (descending): **quando cresce lo stack, gli indirizzi diminuiscono**.
 - Il registro SP **punta alla locazione occupata più in alto nello stack** (cioè l'ultimo elemento inserito) – **full descending**
 - Quando si fa un push, lo stack (SP) viene decrementato e si scrive nella locazione puntata da SP.
 - Quando si fa un pop, si legge il valore puntato dallo stack (SP) e si incrementa il valore di SP.

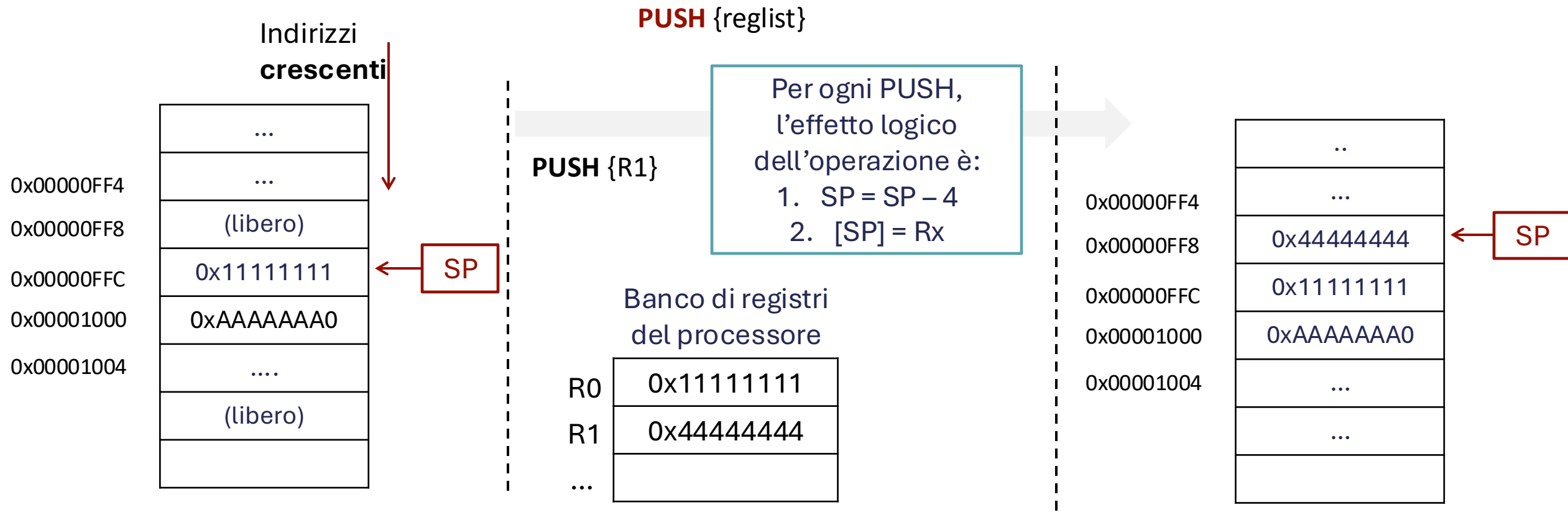
LO STACK: PSEUDO-ISTRUZIONE PUSH (1)

- Per fare il push/pop di valori su/da stack, in assembly GAS sono disponibili due pseudo-istruzioni che l'assembler traduce con altre istruzioni (STM/LDM).



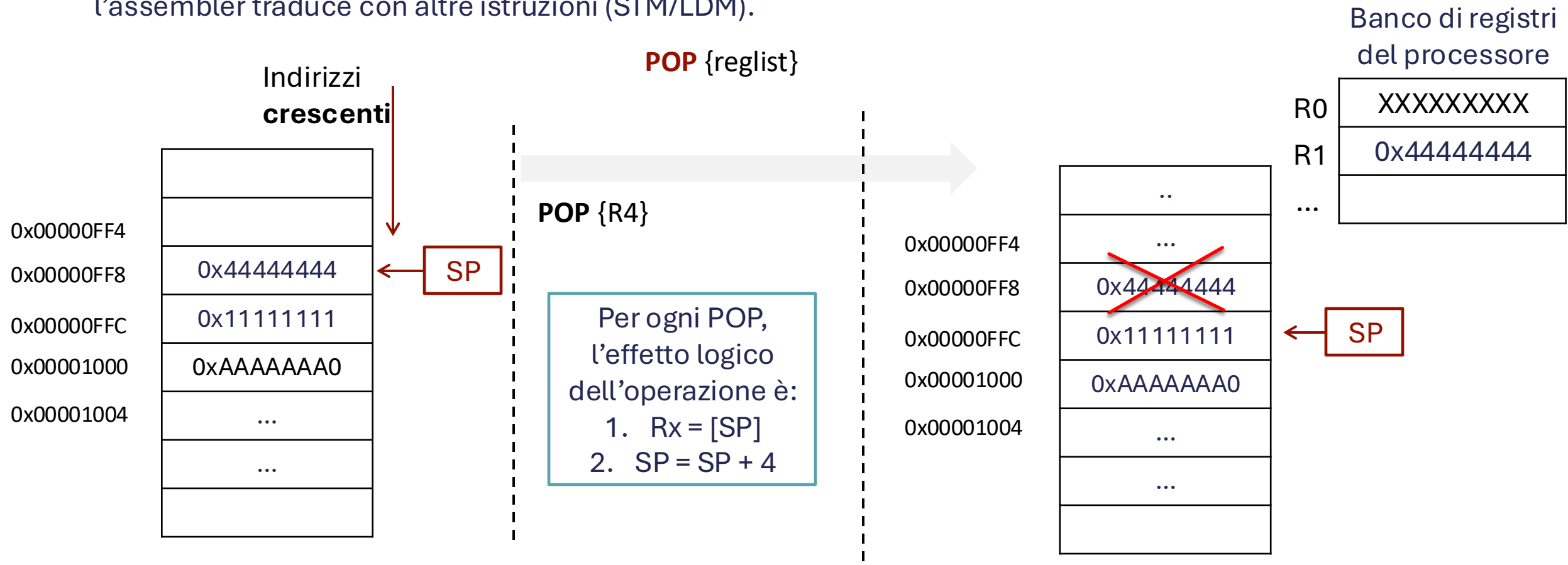
LO STACK: PSEUDO-ISTRUZIONE PUSH (2)

- Per fare il push/pop di valori su/da stack, in assembly GAS sono disponibili due pseudo-istruzioni che l'assembler traduce con altre istruzioni (STM/LDM).



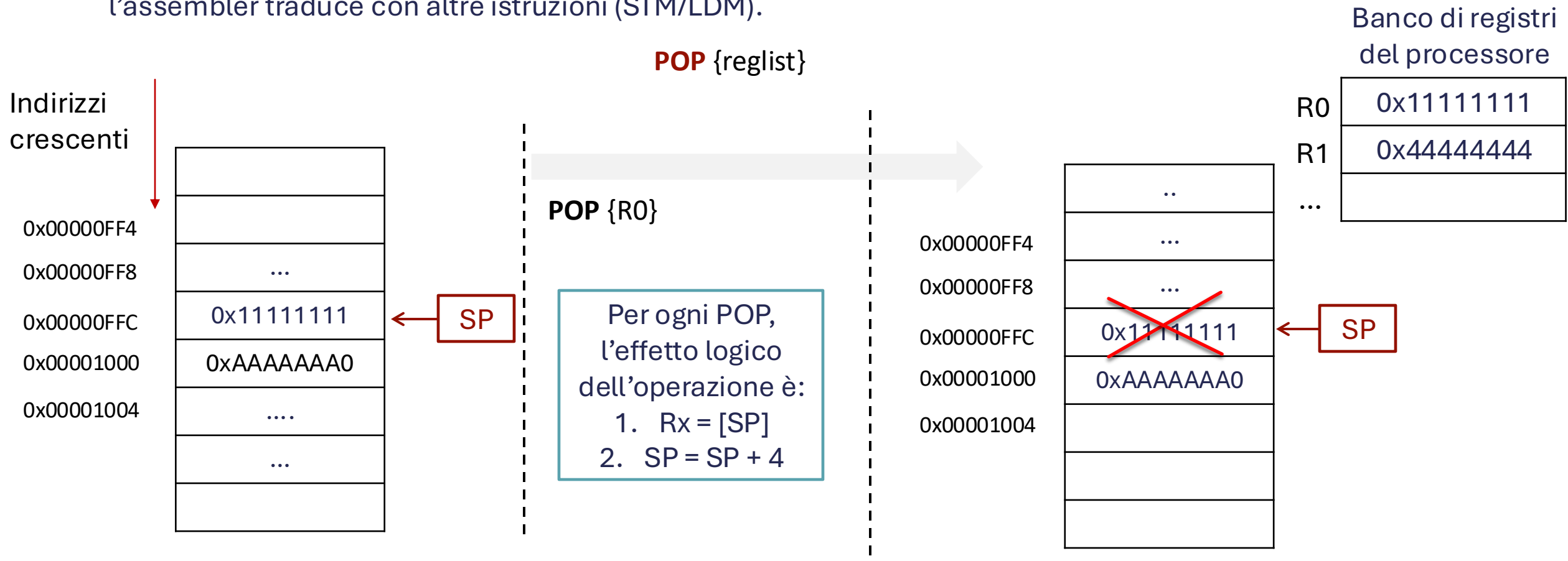
LO STACK: PSEUDO-ISTRUZIONE POP (1)

- Per fare il push/pop di valori su/da stack, in assembly GAS sono disponibili due pseudo-istruzioni che l'assembler traduce con altre istruzioni (STM/LDM).



LO STACK: PSEUDO-ISTRUZIONE POP (2)

- Per fare il push/pop di valori su/da stack, in assembly GAS sono disponibili due pseudo-istruzioni che l'assembler traduce con altre istruzioni (STM/LDM).



PASSAGGIO DI PARAMETRI SU STACK: PASSI

- **PROGRAMMA CHIAMANTE:**
 - Riserva spazio sullo stack per i parametri di output
 - Pone sullo stack i parametri di input
 - Salta alla subroutine (con BL l'indirizzo di ritorno viene salvato in LR)
 - Preleva i parametri di output dallo stack
 - Ripulisce lo stack (può farlo anche la subroutine)
- **SUBROUTINE:**
 - Preleva i parametri di input dallo stack (utilizzando gli offset relativi allo stack pointer) e li copia nei registri
 - Effettua la propria elaborazione e calcola i parametri di output
 - Pone i parametri di output sullo stack nelle posizioni per essi riservate (utilizzando gli offset relativi allo SP)
 - Ripulisce lo stack (opzionale, può farlo anche il main)
 - Salta all'indirizzo di ritorno (MOV PC,LR).

PASSAGGIO DI PARAMETRI SU STACK: CALCOLO DEGLI SPIAZZAMENTI

Programma chiamante

```
MOV    R0,#3
MOV    R1,#4
SUB    SP,SP,#4
PUSH   {R0,R1}
BL     somma
ADD    SP,SP,#8
POP    {R2}
SWI    0x11
```

Subroutine

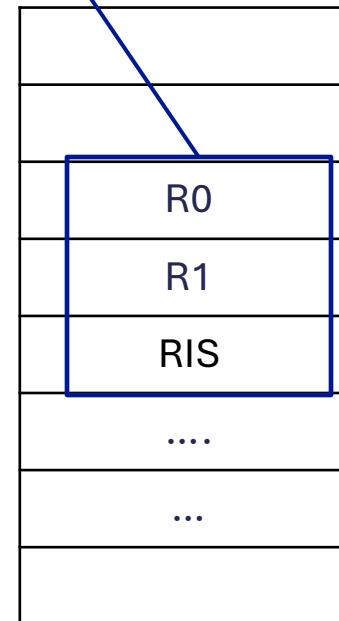
```
somma:  LDR  R0,[SP]
        LDR  R1,[SP,#4]
        ADD  R0, R0, R1
        STR  R0,[SP, #8]
        MOV  PC, LR
```

Prima della
SUB, SP punta
alla base
dello stack

SP_prima

record di attivazione
della funzione
somma

0x000113F0
0x000113F4
0x000113F8
0x000113FC
0x00011400



Dopo la sub e la push,
SP punta alla locazione
dove c'è ultimo valore
inserito (3)

SP_dopo

0
4
8

Gli **spiazzamenti** vanno
calcolati come **costanti**
relative al valore di SP. Ad
esempio lo spazio
riservato per salvare il
risultato si trova ad:
SP + 8

REGISTRO FRAME POINTER

- Se nel corso di esecuzione di una subroutine lo stack pointer subisce delle modifiche (ad esempio, altri dati sono aggiunti dalla subroutine sulla cima dello stack), **lo spiazamento tra il record di attivazione e la cima dello stack puntata da SP non è più costante.**
- È necessario che il programmatore fissi un riferimento che si mantenga costante per tutta l'esecuzione della subroutine, si utilizza il registro **Frame Pointer** o **FP** (R11).
 1. Il valore di FP all'entrata della subroutine dovrà essere innanzitutto salvato sulla cima dello stack con un'istruzione di PUSH.
 2. Successivamente il valore fp sarà caricato con il valore corrente di SP.
 3. Lo **spiazamento** dei campi del record di attivazione **dovrà essere calcolato rispetto all'indirizzo puntato da FP** e si manterrà costante durante tutta l'esecuzione della subroutine.
 4. All'uscita dalla subroutine, il valore precedente di FP sarà ripristinato prelevandolo dalla cima dello stack con l'istruzione di POP, subito prima di effettuare il ritorno al programma chiamante con l'istruzione di MOV PC,LR.

ESEMPIO: CHIAMATA A SOTTOPROGRAMMA E USO DEL FRAME POINTER (1)

```
.text
.global _start
_start: MOV R0,#3
        MOV R1,#4
        SUB SP,SP,#4
        PUSH {R0,R1}
        BL somma
        ADD SP,SP,#8
        POP {R3}
        SWI 0x11
```

1. È buona norma fare il push su stack del valore di FP (che sarà poi aggiornato al valore di SP), ma **anche di LR** (che contiene l'indirizzo di ritorno al chiamante). Questo permette anche la possibilità di *chiamate innestate*.

```
somma: PUSH {FP,LR}
        MOV FP,SP
        PUSH {reglist}
        LDR R0,[FP,#8]
        LDR R1,[FP,#12]
        ADD R0,R0,R1
        STR R0,[FP,#16]
        POP {reglist}
        POP {FP,LR}
        MOV PC,LR
```

new_FP

0x000113F0
0x000113F4
0x000113F8
0x000113FC
0x00011400

OLD_FP

OLD_LR

R0

R1

SPAZIO_RIS

....

0
4
8
12
16

SP

2. È buona norma fare il push su stack dei registri che saranno «sporcati» (utilizzati all'interno del sottoprogramma (`PUSH {reglist}`)). Al termine delle elaborazioni del sottoprogramma, si ripristina il contenuto dei registri (`POP {reglist}`). Questo permette al programma chiamante, al rientro dalla chiamata a sottoprogramma, di trovare nei registri i valori che c'erano esattamente prima della chiamata.

3. Gli spiazamenti sono calcolati a partire dal new_FP che è costante, mentre SP può essere eventualmente modificato

ESEMPIO: CHIAMATA A SOTTOPROGRAMMA E USO DEL FRAME POINTER (2)

```
.text
.global _start
_start: MOV R0,#3
        MOV R1,#4
        SUB SP,SP,#4
        PUSH {R0,R1}
        BL somma
        ADD SP,SP,#8
        POP {R3}
        SWI 0x11
```

```
somma: PUSH {FP,LR}
        MOV FP,SP
        PUSH {reglist}
        LDR R0,[FP,#8]
        LDR R1,[FP,#12]
        ADD R0,R0,R1
        STR R0,[FP,#16]
        POP {reglist}
        POP {FP,LR}
        MOV PC,LR
```

0x000113F0
0x000113F4
0x000113F8
0x000113FC
0x00011400



SP

Al momento della POP, il frame pointer viene ripristinato al vecchio (OLD_FP), il LR viene ripristinato al vecchio valore (OLD_LR). La pop ha l'effetto di eliminare questi elementi dallo stack. Quindi SP punterà alla cella in cui è contenuto R0.

ESEMPIO: CHIAMATA A SOTTOPROGRAMMA E USO DEL FRAME POINTER (2)

```
.text
.global _start
_start: MOV R0,#3
        MOV R1,#4
        SUB SP,SP,#4
        PUSH {R0,R1}
        BL somma
        ADD SP,SP,#8
        POP {R3}
        SWI 0x11
```

```
somma: PUSH {FP,LR}
        MOV FP,SP
        PUSH {reglist}
        LDR R0,[FP,#8]
        LDR R1,[FP,#12]
        ADD R0,R0,R1
        STR R0,[FP,#16]
        POP {reglist}
        POP {FP,LR}
        MOV PC,LR
```

0x000113F0
0x000113F4
0x000113F8
0x000113FC
0x00011400

OLD_FP
OLD_LR
R0
R1
SPAZIO_RIS
...
...

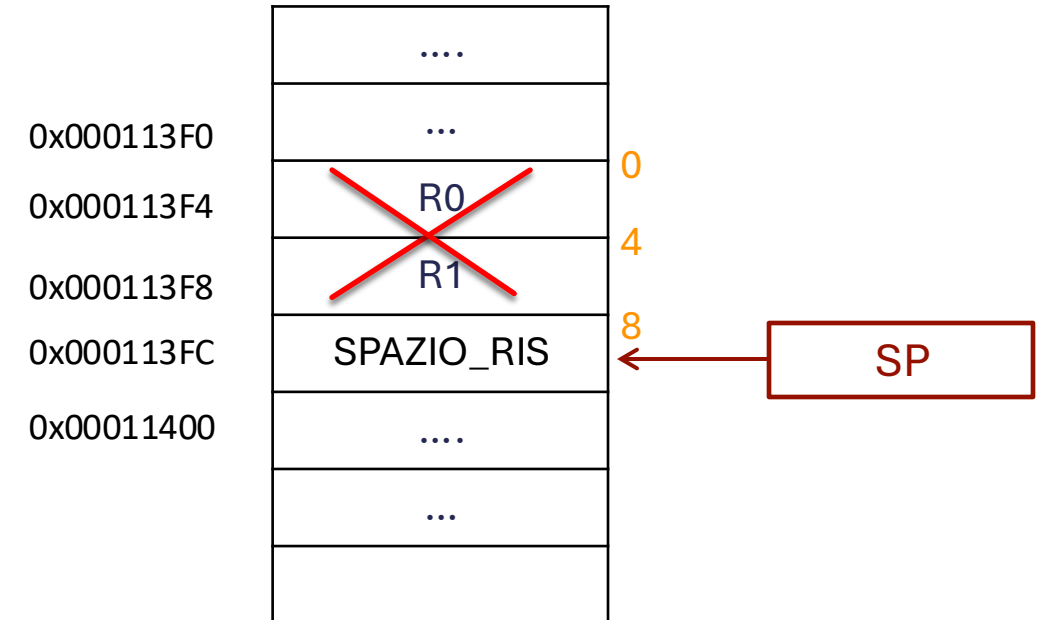
SP

Con questa istruzione si ritorna al programma chiamante, ripristinando in PC il valore LR (l'indirizzo di ritorno, ovvero dell'istruzione ADD che è quella immediatamente successiva alla chiamata a sottoprogramma).

ESEMPIO: CHIAMATA A SOTTOPROGRAMMA E USO DEL FRAME POINTER (2)

```
.text
.global _start
_start: MOV R0,#3
        MOV R1,#4
        SUB SP,SP,#4
        PUSH {R0,R1}
        BL somma
        ADD SP,SP,#8
        POP {R3}
        SWI 0x11
```

```
somma: PUSH {FP,LR}
        MOV FP,SP
        LDR R0,[FP,#8]
        LDR R1,[FP,#12]
        ADD R0,R0,R1
        STR R0,[FP,#16]
        POP {FP,LR}
        MOV PC,LR
```



Quando si rientra nel programma chiamante, SP punta ad R0. Gli **spiazzamenti (per pulire stack e recupero del risultato) andranno calcolati rispetto al valore di SP al rientro dal sottoprogramma!!** In questo caso, l'istruzione ha l'effetto di incrementare SP di 8 locazioni per puntare alla cella in cui è riservato lo spazio del risultato (precedentemente salvato dalla subroutine), così da poter fare la successiva POP.